

Lightweight convolutional neural network of YOLO v3-Tiny algorithm on FPGA for target detection

^{1,2} Jitong Xin

¹School of Physical Science and Technology, Lanzhou University

²Institute of Microelectronics, School of Physics Science and Technology

Lanzhou, China

2579379665@qq.com

^{1,2} LuoJia Shi

¹School of Physical Science and Technology, Lanzhou University

²Institute of Microelectronics, School of Physics Science and Technology

Lanzhou, China

1135820276@qq.com

^{1,2} Hairong Li

¹School of Physical Science and Technology, Lanzhou University

²Institute of Microelectronics, School of Physics Science and Technology

Lanzhou, China

hrli@lzu.edu.cn

^{1,2,*} Peng Wang

¹School of Physical Science and Technology, Lanzhou University

²Key Laboratory for Magnetism and Magnetic Materials of the Ministry of Education

Lanzhou, China

wangpeng@lzu.edu.cn

*Corresponding author

^{1,2} Meiyi Cha

¹School of Physical Science and Technology, Lanzhou University

²Institute of Microelectronics, School of Physics Science and Technology

Lanzhou, China

894462129@qq.com

^{1,2} Chunyu Long

¹School of Physical Science and Technology, Lanzhou University

²Institute of Microelectronics, School of Physics Science and Technology

Lanzhou, China

1052779873@qq.com

^{1,2} Fangcong Wang

¹School of Physical Science and Technology, Lanzhou University

²Institute of Microelectronics, School of Physics Science and Technology

Lanzhou, China

wangfc@lzu.edu.cn

Abstract—With the rapid development of artificial intelligence, the convolution of the neural network has been deploying across a range of industries, and has certain applications in national defense security, transportation monitoring and medical research. However, due to the constraints of speed and the power consumption, convolutional neural networks are still limited to a great extent in the implementation of the edge computing and mobile devices, etc. For this reason, we design a lightweight convolutional neural network based on FPGA. In this paper, we use the YOLOv3-Tiny algorithm, which is fast in execution, small in computation and small in size, is suitable for deployments on embedded devices such as FPGA. This paper uses 16-bit fixed-point quantization, special data storage and hardware circuit for convolutional computation written in verilog - hardware description language, consumes 512 DSPs, consumes 37.037 ms to recognize a single image frame, and consumes 9.611W. The system basically achieves the design goal of target detection.

Keywords—FPGA, target detection, YOLOv3-Tiny, verilog

I. INTRODUCTION

With the rapid development in the field of computer vision, neural network plays an increasingly important role in national production life. Target detection technology, as a direction of computer vision, has a broad application prospect in many fields of life, such as monitoring pedestrian detection on roads, driverless cars, medical image analysis, drone navigation and national defense security. The two main tasks of target detection are to locate the location of a target from a scene or image and to identify the class of the target. However, in most practical applications, target detection technology needs to port to embedded devices, and the current market mainly uses GPU processors, which is much more efficient than CPU processors, but their power consumption is too high and expensive to be widely applicable to various environments [1]. The CPU processor consumes less power than the GPU, but the processing speed is too slow. FPGA, as a field-programmable logic device, is undoubtedly a good choice for implementing target detection algorithms from the point of view of low power consumption and high speed. FPGA has the advantages of low power

consumption, speed and high flexibility, and is now widely used to accelerate the forward inference process of CNN.

YOLO detection algorithm is currently mainly used in the field of target detection, YOLO algorithm is proposed by Joseph Redmond and Ali Farhadi, YOLO algorithm creatively combines two stages of candidate area and target identification into one which can complete the target detection, also achieve the end-to-end target detection [2]. Since the conventional YOLO algorithm is too large for deployment to embedded devices, the YOLOv3-Tiny algorithm is undoubtedly a better choice [3].

This paper adopts 16-bit fixed-point quantization according to the characteristics of FPGA, and writes the convolution unit, graphics pre-processing unit, image post-processing unit and data storage unit by verilog, in which the convolution module adopts full parallel mode to improve the calculation speed, and the data storage part adopts DDR3 partition storage to make full use of the storage space and greatly shorten the recognition time.

II. ALGORITHM SELECTION

A. Algorithm comparison

Before we start designing a lightweight FPGA-based convolutional neural network, we need to select a lightweight and efficient target detection algorithm for the system. There are two ideas of target detection algorithms based on deep learning techniques: two-stage target detection algorithm and one-stage target detection algorithm. The two-stage target detection algorithm is based on the principle of selecting the region where the target may exist and then further classifying and regressing the selected region, which has a high detection accuracy but a slow detection speed, represented by algorithms such as FAST-RCNN [3]. A stage target detection algorithm treats the detection problem as a regression problem, the input image directly through the convolutional network to regress the coordinates and category of the target, the speed for the detection is faster, but the detection accuracy is lower, the representative algorithms are YOLO, SSD, YOLOv3-Tiny, etc [4]. Table I shows the detection effect of several current detection algorithms, where AP is the average accuracy, the larger the value is the detection better the effect, and the less the detection time the faster the system. In order to better complete the target detection of real-time video images, we must choose a fast detection algorithm. It can be seen from the table that the detection speed of YOLOv3 and YOLOv3-Tiny is faster than other algorithms. In comparison between YOLOv3 and YOLOv3-Tiny, although YOLOv3 has a higher AP value, YOLOv3-Tiny is smaller than YOLOv3. The smaller size makes YOLOv3-Tiny more suitable for deployment on embedded devices. In summary, this article finally uses the YOLOv3-Tiny algorithm.

TABLE I. COMPARISON OF THE EFFICIENCY OF EACH ALGORITHM

Algorithm abbreviation	Detection effect (AP Value)	Testing time(ms)
YOLOv3 ^[5]	0.787	51
YOLOv3-Tiny ^[5]	0.722	4
SSD ^[6]	0.776	61
DSSD ^[7]	0.782	85
RetinaNet ^[8]	0.894	198
R-FCN ^[9]	0.865	172
M2Det ^[10]	0.875	165

B. YOLOv3-Tiny algorithm model

YOLO v3-Tiny is a simplified version of YOLO v3, which is a lightweight network. The input RGB image is firstly used to obtain two feature maps of different scales after passing through the feature extraction network, and then the input image is divided into grid cells of the same size as the feature maps, and the grid which located in the center of each object of the image is responsible for the detection of that object, i.e., the grid which contains the center coordinates of an object is responsible for predicting the object followed by three predictions per cell grid. The final output prediction map has two dimensions, i.e., the dimension of feature extraction, such as the 13 × 13 feature map, and the second dimension is depth. As in the previous YOLO version, YOLOv3-Tiny also uses the leaky relu function, which corrects the data distribution by adding constant and multiplication of the negative axis data so that all negative axis information is not lost[11].

C. Model structure

The network structure of YOLOv3-Tiny network is shown in Fig. 1. Similar to YOLO v3, two sizes of convolutional kernels, i.e., 3*3 and 1*1, are used in the network. The 3*3 convolutional kernels are mainly used to extract feature information of images, and the 1*1 convolutional kernels can add nonlinear features and reduce the parameters for network training while keeping the feature map size unchanged. YOLOv3-Tiny backbone network mainly has seven 1*1 convolutional layers and six maximum pooling layers, the first five maximum pooling layers have a step size of 2 and the last one has a step size of 1. When the input size is 416*416 images, the output of the overall network is a 13*13 feature map. After upsampling, the output is tensor spliced with the output of the 5th convolutional layer and then convolutional operation is performed.

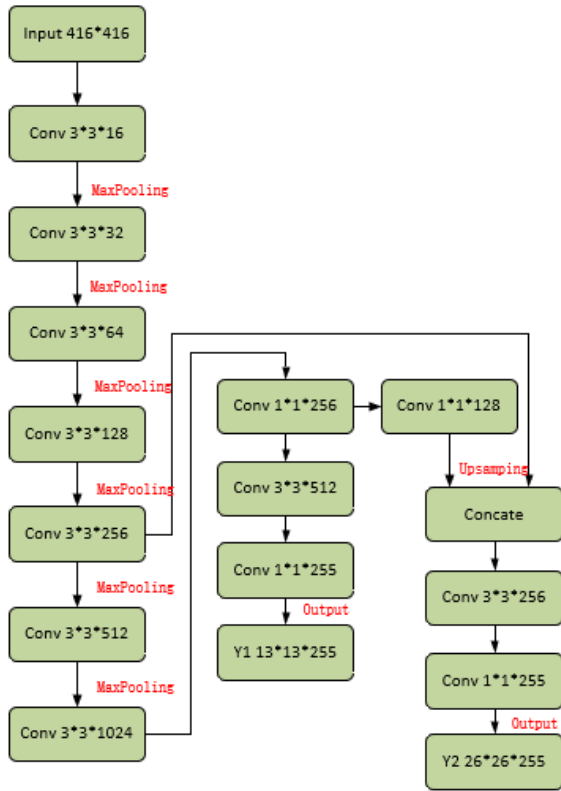


Figure 1. Network structure of the YOLOv3-Tiny network.

III. SYSTEM IMPLEMENTATION

A. Overall program

See Fig. 2 for the structure of this design. The whole accelerator is divided into image preprocessing unit, convolution unit, image post-processing unit, DDR3 storage unit and PCIE unit. The image pre-processing unit mainly controls the reading and writing of image data, weight data and bias data, and the convolution unit mainly completes the convolution and relu operations[12]. The image post-processing module mainly completes maxpool and upsample operations. The DDR3 storage unit mainly completes the storage of image data, weight data and bias data. The PCIE unit mainly completes the communication with the PC.

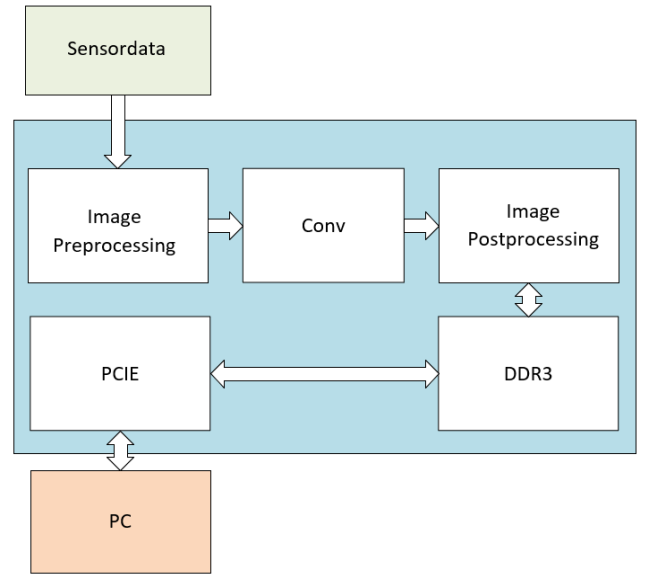


Figure 2. Overall design structure diagram.

The specific workflow of the system is as follows: after the initialization, the weight data and bias data are first transferred from the PC side to the DDR3 side through the PCIE unit for storage. The image first enters from the camera module, enters the convolution unit, performs the convolution operation and relu operation, and the calculated result then enters the image post-processing unit to complete the max pool operation, after which the output result is stored in the DDR3 to complete the first layer operation, and then the second layer operation starts, first the output data of the first layer and the required weight and bias data are taken out from the DDR3, and continue to pass into the convolution. The next operation of each layer is similar to the operation of the first layer. Finally, the system passes the results of layer10 and layer13 back to the PC for post-processing of the data to obtain the final object detection frame and object detection class. The above operations are mainly shown in Table II.

As shown in the Table II, this table is an image data allocation table for each layer of DDR3 storage, where m0, m1, m2 and m3 are four storage spaces with different addresses for storing image data in DDR3.

TABLE II. DDR3 STORAGE IMAGE DATA ALLOCATION

Layers	Read_image0	Read_image1	Result0	Result1
Layer1	0	0	m0	0
Layer2	m0	0	m1	0
Layer3	m1	0	m0	0
Layer4	m0	0	m1	0
Layer5	m1	0	m0	m3
Layer6	m0	0	m1	0
Layer7	m1	0	m0	0
Layer8	m0	0	m2	0
Layer9	m2	0	m0	0

Layer10	m0	0	m1	0
Layer11	m2	0	m0	0
Layer12	m0	m3	m1	0
Layer13	m1	0	m0	0

B. Convolutional module design

The convolution module is essentially performing multiplication and addition operations, so the efficiency of the convolution module directly affects the efficiency of the entire accelerator. So how to improve the efficiency of multiplication computation is crucial. Before proceeding with the design of the convolution module, we first use the 32-bit weight data trained by the darknet framework through Caffe-Ristretto (Caffe-Ristretto is an automatic CNN quantization tool that compresses 32-bit floating-point networks.) Perform fixed-point quantization. The 16-bit fixed-point weight data is obtained, and its size is 34.2 MB.

The fixed-point matrix multiplication unit designed in this paper consists of multiple PE basic units, which is computed in parallel to reduce the computational complexity. Each PE (Processing Elements) unit contains a fixed-point multiplication calculation unit, a bias addition calculation unit, and a relu function calculation as well as a storage module to store the calculation results. As shown in the Fig. 3, the weight data, image data, and bias data are input to the PE unit. In the PE unit. The PE unit mainly completes the multiplication operation and the addition operation, it also needs to do a shift operation on the result of the multiplication and addition operation to ensure the number of output bits. After the multiplication and addition operation, the result is added with the bias data and shifted again to output the final result.

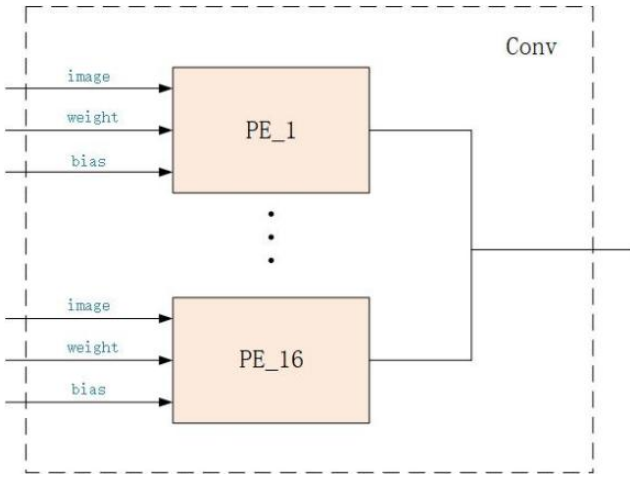


Figure 3. The diagram of convolution module.

In the design of the fixed-point multiplication calculation unit of a PE, we split the multiplication calculation, as shown in Fig. 4. We split the large bit-width multiplication calculation of 512-bit input data into thirty-two 16-bit multiplication calculations. After the multiplication calculation, the result enters the Sum unit and is continuously added to get the final result.

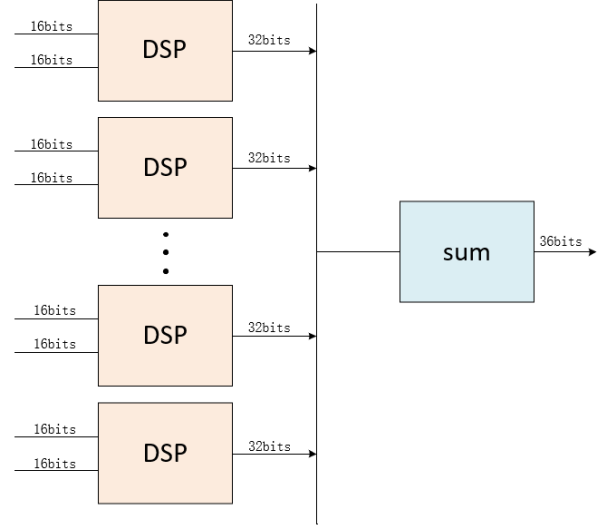


Figure 4. PE unit structure diagram.

As shown in the Fig. 5, the sum unit is composed of 5 layers of adders. The data first completes an addition operation through 16 adders, and then the calculation results are continuously added, and finally the final result is obtained.

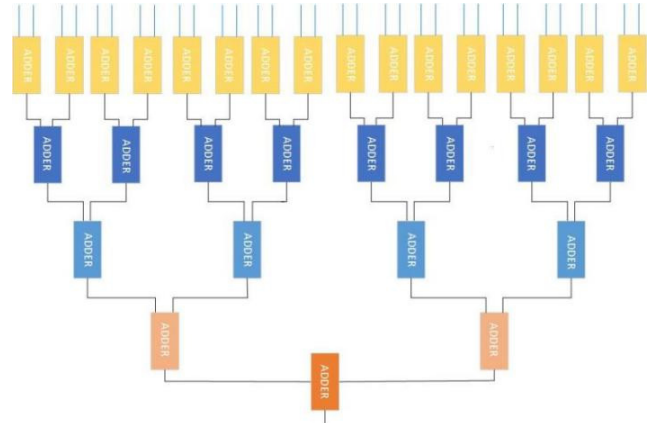


Figure 5. Sum unit structure diagram.

For the relu function calculation unit, the general YOLOv3-Tiny network uses the leaky relu as the activation function, where the leaky relu is represented by Equation (1). However, when $x < 0$, $Y = kx$, where k is a decimal number between 0 and 1. Thus, the FPGA needs to call DSP resources for floating-point multiplication calculation, which not only wastes resources but also consumes time. In order to reduce resource consumption and save time, we choose using the Relu function as the activation function. Relu function formula is expressed in Equation (2)[2].

$$Y = \begin{cases} x, & x \geq 0 \\ kx, & x < 0 \end{cases} \quad (1)$$

$$Y = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (2)$$

Finally, the convolution module design is completed by designing the convolution module in verilog. The simulation of the convolution module is shown in Fig. 6.

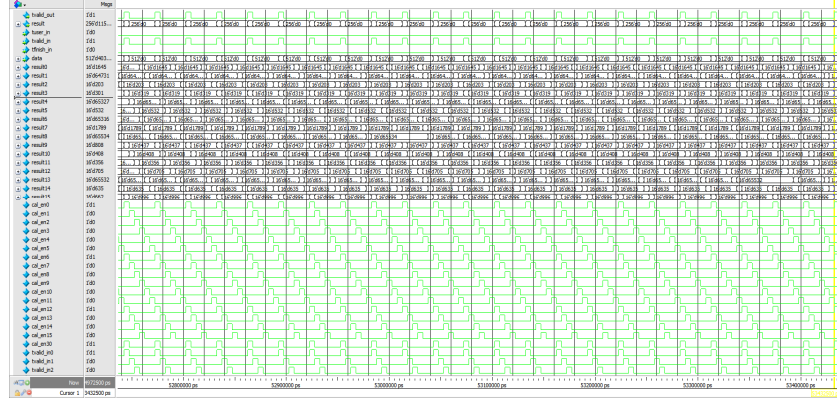


Figure 6. Simulation diagram of convolution module.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental results

The FPGA used in this experiment is Xilinx's Kintex-7 series development board, and the main chip is XC7K325T with 840 DSP resources and 1GB 32-bit DDR3 memory particles. The hardware development platform used for the experiments is Vivado 2019.2, and the simulation software is ModelSim SE-64 10.5. The official coco dataset is used for the data set, which is trained and quantified by the software, and the 16-bit weights and bias data obtained are applied to the inference of the accelerator. The accelerator part is written entirely in verilog. The recognition results are shown in Fig. 7. The figure below shows the result that the person in the image has been boxed in and the platform can achieve the target detection function.

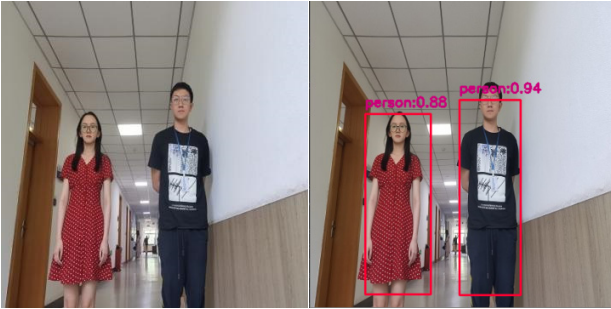


Figure 7. Identification effect map.

B. Comparison with General CPU Performance

We also tested the same pictures on CPU. We compare FPGA and CPU in terms of recognition of single picture time, frequency and power consumption of a single picture[13]. As shown in Table III, the performance of the FPGA is far superior to that of the CPU in terms of speed and power consumption, with 13.7% of the power consumption and 4.3 times the speed of the CPU. The performance of the FPGA can meet the basic requirements of the target detection for real-time video.

TABLE III. COMPARISON OF CPU AND FPGA

CPU			FPGA		
Intel Core i5 4440			Xilinx XC7k325T		
Recognition of single picture time (ms)	Power Consum -ption (W)	Frequency (MHz)	Recognition of single picture time (ms)	Power Consum -ption (W)	Frequency (MHz)
160	70	4000	37.037	9.6	100

V. CONCLUSION AND OUTLOOK

A. Conclusion

In this paper, we designed a lightweight neural network target detection system based on FPGA. Firstly, Through comparison with different kinds of algorithms, we employ YOLOv3-Tiny algorithm. Then, we trained YOLOv3-Tiny and quantized the resulting weight file with bias data at 16-bit; secondly, we designed the circuit of YOLOv3-Tiny on this paper using the verilog. Finally, we completed the data post-processing part on the PC side. Through the experiment, the system can realize the target detection. The system operates at 100MHz, consumes 9.611W, and takes about 37.037ms to recognize a single image. The speed is 4 times faster than the general CPU and the power consumption is 13.73 % of the general CPU. With the advantages of high speed and low power consumption, it is beneficial to be deployed in portable devices and real-time video processing systems.

B. Outlook

The system described in this paper can be widely used in scenarios such as driverless, drone, and edge computing, giving full play to the advantages of fast speed, good real-time, and low power consumption. However, there is still much room for optimization in the system described in this paper. For example, 8-bit quantization can be continued, and also ZYNQ which can integrate the data post-processing in the ARM side to make the overall algorithm more integrated. Secondly, it still causes missed recognition when recognizing multiple target objects, so it is also necessary to make improvements to the algorithm itself at a later stage to balance the relationship between resource

consumption, speed, and power consumption as well as accuracy, so that the overall cost performance of the system can be improved.

REFERENCES

- [1] Qiu, J Wang, S Yao et al., "Going Deeper with Embedded FPGA Platform for Convolutional Neural Network [C]", Acm/sigda International Symposium on Field-programmable Gate Arrays, 2016.
- [2] Zhang, J. Cao, Q. Zhang, Q. Zhang, Y. Zhang and Y. Wang, "An FPGA-Based Reconfigurable CNN Accelerator for YOLO," 2020 IEEE 3rd International Conference on Electronics Technology (ICET), pp. 74-78, 2020.
- [3] Girshick, R.B.: Fast R-CNN. in: ICCV. (2015) 1440-1448 .
- [4] ZHANG Y, SHEN Y L, ZHANG J. An improved tiny-yolov3 pedes-trian detection algorithm [J]. Optik, 2019, 183: 17-23.
- [5] Redmon J, Farhadi A. Yolov3: An incremental improvement[J]. arXiv preprint arXiv:1804.02767, 2018.
- [6] Liu W, Anguelov D, Erhan D, et al. Ssd: Single shot multibox detector[C]. European conference on computer vision. Springer, Cham, 2016: 21-37.
- [7] Fu C Y, Liu W, Ranga A, et al. DSSD: Deconvolutional Single Shot Detector[J]. 2017.
- [8] DAI J, LI Y, HE K, et al. R-FCN: object detection via region-based fully convolutional networks [C]. 29th Conference on Neural Information Processing Systems, 2016: 379-387.
- [9] Lin T Y, Goyal P, Girshick R, et al. Focal Loss for Dense Object Detection[C]. 2017 IEEE International Conference on Computer Vision (ICCV). IEEE, 2017:2999-3007.
- [10] Zhao Q, Sheng T, Wang Y, et al. M2Det: A Single-Shot Object Detector based on Multi-Level Feature Pyramid Network. 2018.
- [11] "Pattern Recognition and Computer Vision", Springer Science and Business Media LLC, 2020.
- [12] Advances in Intelligent Analysis of Medical Data and Decision Support Systems", Springer Science and Business Media LLC, 2013.
- [13] Fanghong Bi, Jun Yang. "Target Detection System Design and FPGA Implementation Based on YOLO v2 Algorithm", 2019 3rd International Conference on Imaging, Signal Processing and Communication (ICISPC), 2019.